



**UNIVERSIDAD DE
COSTA RICA**
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Laboratorio 8: Integración Pila Completa (Full-Stack) - Consola de Operación Logística "ExpresoFast" con Backend REST, HTML5 Semántico y CSS3 Responsivo

Metadatos

- **Tiempo Estimado:** 6 horas (trabajo autónomo individual)

Herramientas Requeridas:

- Java 21 (o versión local instalada) con Maven.
- Spring Boot 3.x (Spring Data JPA, Spring Security, JWT).
- Navegador Web moderno (Google Chrome / Firefox) con DevTools.

- Editor de código (Visual Studio Code).
- Servidor web de desarrollo local (Live Server o extensión equivalente).
- Git y Cuenta activa de GitHub.

Metas de Aprendizaje:

1. Conectar la API RESTful de Spring Boot desarrollada y probada en los Laboratorios 6 y 7 con una aplicación web cliente en el front-end.
2. Diseñar la estructura cliente utilizando marcado semántico HTML5 (`<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>`, `<footer>`) y estándares de accesibilidad universal (WCAG).
3. Construir un sistema de estilos CSS3 modular con variables CSS (`:root`), layouts adaptativos con Flexbox y CSS Grid Layout, y respuestas responsivas (*Mobile-First*).
4. Implementar la interactividad asíncrona mediante JavaScript (Fetch API / `async-await`) para autenticar usuarios, gestionar tokens JWT en la cabecera `Authorization` y consumir los endpoints protegidos.
5. Renderizar paneles dinámicos de control y tarjetas de estado de envíos según el rol del usuario autenticado (`ROLE_ADMIN`, `ROLE_OPERADOR`, `ROLE_CONDUCTOR`).

Introducción y Caso de Estudio (Cuarta Parte)

En las entregas anteriores de la plataforma **ExpresoFast** (Laboratorios 5, 6 y 7), su equipo construyó la capa de persistencia relacional con Spring Data JPA, aseguró las operaciones mediante JSON Web Tokens (JWT) y control de acceso RBAC, estandarizó el manejo de excepciones bajo el protocolo RFC 7807 y certificó la suite de pruebas unitarias con JUnit 5 y Mockito.

Para esta **Cuarta Parte**, la Dirección Ejecutiva de ExpresoFast solicita desplegar la **Consola Web de Operación Logística**. Esta interfaz cliente permitirá a los distintos actores de la compañía interactuar de forma intuitiva con el sistema desde cualquier dispositivo (computadoras de escritorio, tabletas y teléfonos móviles).

El sistema cliente debe consumir de forma transparente los servicios web RESTful del back-end, gestionando de forma segura la sesión del usuario y ofreciendo una experiencia de usuario (UX) fluida, responsiva y accesible.

Requerimientos Técnicos del Laboratorio

1. Configuración de CORS en el Back-end (Spring Boot)

Para permitir que la aplicación cliente en el front-end realice peticiones asíncronas hacia la API REST sin ser bloqueada por el navegador, configure el soporte de **Intercambio de Recursos de Origen Cruzado (CORS)** en su clase de configuración de Spring Security o en los controladores REST:

```
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
            .allowedOrigins("http://localhost:5500", "http://127.0.0.1:5500")
            .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
            .allowedHeaders("")
            .allowCredentials(true);
    }
}
```

2. Estructura Cliente Semántica HTML5 (`index.html` y `dashboard.html`)

Usted debe estructurar dos vistas principales utilizando marcado semántico estricto HTML5 sin hacer uso exclusivo de elementos genéricos `<div>`:

- **[] Vista 1: Autenticación (`index.html`):**
 - Un formulario accesible de inicio de sesión envuelto en la etiqueta `<form id="loginForm">`.
 - Etiquetas `<label>` vinculadas mediante el atributo `for` a cada campo de texto (`<input type="text" id="username">` y `<input type="password" id="password">`).

Mensajes de error accesibles para notificar fallos de credenciales con atributos ARIA (`aria-live="polite"`).

[] Vista 2: Consola de Operaciones (`dashboard.html`):

- `<header>`: Contiene el logotipo de ExpresoFast, el nombre del usuario autenticado y el botón para cerrar sesión (*Logout*).
- `<nav>`: Menú de navegación principal para filtrar envíos por estado (Todos, Pendientes, En Tránsito, Entregados).

`<main>`: Sección principal dividida en:

- `<section id="kpiSection">`: Tarjetas de indicadores clave (Total Envíos, Vehículos Activos, Paquetes Entregados).
- `<section id="enviosGrid">`: Rejilla principal donde se renderizan las tarjetas `<article>` de cada envío.
- `<aside>`: Panel lateral reservado para la Bitácora de Auditoría (visible únicamente para el `ROLE_ADMIN`).
- `<footer>`: Pie de página institucional con derechos de autor y enlaces de soporte.

3. Sistema de Estilos Responsivo CSS3 (`styles.css`)

Construya una hoja de estilos modular utilizando CSS3 moderno:

- **Variables CSS (`:root`)**: Defina paletas de color primarias, secundarias, neutras, tipografías, sombras y radios de borde:

```
:root {  
  --primary-color: #0284c7;  
  --primary-dark: #0369a1;  
  --bg-dark: #0f172a;  
  --bg-card: #1e293b;  
  --text-main: #f8fafc;  
  --text-muted: #94a3b8;  
  --accent-success: #22c55e;
```

```
--accent-warning: #eab308;
--font-family: 'Inter', system-ui, sans-serif;
}
```

Layouts con CSS Grid y Flexbox:

- Utilice **CSS Grid** (`grid-template-columns: repeat(auto-fit, minmax(280px, 1fr))`) para la rejilla adaptativa de tarjetas de envíos.
- Utilice **Flexbox** para centrar formularios, alinear los elementos de la barra de navegación y distribuir los contenidos internos de cada tarjeta.
- **Diseño Adaptativo (*Media Queries*)**: Garantice que en pantallas móviles ($\leq 768\text{px}$) la barra lateral `<aside>` y el menú de navegación se adapten verticalmente sin desbordar la pantalla.

4. Interactividad Asíncrona con JavaScript (app.js)

Implemente la lógica del cliente utilizando JavaScript moderno (ES6+):

- **[] Inicio de Sesión y Manejo del Token JWT:**
 - Capturar el evento `submit` del formulario en `index.html`.
 - Enviar una petición POST asíncrona mediante `fetch('/api/auth/login')`.
 - Al recibir una respuesta exitosa (HTTP 200 OK), almacenar el token JWT devuelto en `sessionStorage.setItem('jwt_token', token)`.

Redireccionar automáticamente a `dashboard.html`.

[] Consumo de APIs Protegidas:

- En cada petición HTTP asíncrona (`GET /api/envios`, `PUT /api/envios/{id}/estado`), adjunte el encabezado de autorización:

```
const token = sessionStorage.getItem('jwt_token');
```

```
const respuesta = await fetch('/api/envios', {
  method: 'GET',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${token}`
  }
});
```

- [] **Manejo de Errores HTTP:**
- Si la API devuelve un código de estado 401 Unauthorized o 403 Forbidden, limpie el almacenamiento local y redirija al usuario a `index.html`.
- Si la API devuelve 400 Bad Request, extraiga la lista de errores del JSON RFC 7807 y despléguelos dinámicamente en una caja de alerta visual.

5. Renderizado Dinámico Basado en Roles (RBAC Cliente)

Decodifique las autoridades del token JWT en el cliente para adaptar la interfaz web según el rol del usuario:

- **ROLE_ADMIN:** Visualiza la sección lateral `<aside>` con la bitácora de auditoría histórica (BitacoraEnvio) y el botón para registrar nuevos vehículos.
- **ROLE_OPERADOR:** Visualiza los botones de acción para asignar vehículos y actualizar estados de envíos a `EN_TRANSITO`.
- **ROLE_CONDUCTOR:** Solo visualiza las tarjetas de envíos asignados a su vehículo y el botón para marcar la entrega como `ENTREGADO`.

Entregables y Modalidad de Entrega

- **Modalidad:** Trabajo Autónomo Individual.

Repositorio en GitHub:

- Suba el proyecto estructurado en dos carpetas principales: `/backend` (proyecto Spring Boot) y `/frontend` (archivos HTML, CSS y JS).
- Incluya un archivo `README.md` con instrucciones para ejecutar la API REST y desplegar el cliente en un servidor local.

Entrega en Mediación Virtual:

- Suba un documento PDF o archivo comprimido que contenga:
 - Enlace al repositorio público de GitHub.
 - Capturas de pantalla de la pantalla de Login y del Dashboard en modo escritorio y modo móvil.
 - Capturas de la consola DevTools (pestaña *Network*) mostrando las solicitudes HTTP con la cabecera `Authorization: Bearer <token>`.

Rúbrica de Evaluación (Total: 100 Puntos)

Criterio de Evaluación	Puntuación	Descripción
Configuración CORS en Back-end	15 pts	Habilitación y configuración correcta de políticas CORS en Spring Boot permitiendo peticiones asíncronas desde el origen cliente.
Marcado Semántico HTML5 y Accesibilidad	20 pts	Uso estricto de elementos semánticos (<code><header></code> , <code><nav></code> , <code><main></code> , <code><article></code> , <code><aside></code> , <code><footer></code>) y formularios accesibles con <code><label></code> .

Diseño y Estilos CSS3 Responsivos	25 pts	Uso de variables CSS, maquetado con Flexbox y CSS Grid Layout, adaptación responsiva en pantallas móviles y de escritorio.
Consumo de APIs con Fetch API y JWT	25 pts	Peticiones asíncronas async/await correctas, almacenamiento del JWT en sessionStorage y envío de la cabecera <code>Authorization: Bearer.</code>
Manejo de Errores y Vistas por Rol	15 pts	Captura visual de errores HTTP (400, 401, 403, 404) y renderizado condicional de componentes según el rol del usuario (<code>ADMIN</code> , <code>OPERADOR</code> , <code>CONDUCTOR</code>).